

API PASSO A PASSO

O programa é uma pequena **API REST de produtos** escrita somente com a biblioteca padrão do Go. Ela permite consultar, cadastrar, alterar e excluir produtos. Além disso, possui um middleware de log e outro de autenticação.

1. O que esse programa faz

Imagine que temos uma pequena loja com estes produtos:

```
1 - Notebook - R$ 3500
2 - Mouse    - R$ 120
3 - Teclado  - R$ 250
```

A aplicação permite fazer:

```
GET      /produtos
GET      /produtos?id=2

POST     /produtos

PATCH   /produtos?id=2

DELETE   /produtos?id=3
```

Cada método HTTP tem uma função diferente:

Método Significado

```
GET      Consultar
POST     Criar
PATCH   Alterar
DELETE   Excluir
```

O GET pode ser utilizado sem autenticação. Já POST, PATCH e DELETE passam pelo middleware de autenticação.

2. package main

Logo no início temos:

```
package main
```

Todo programa Go executável precisa ter um pacote chamado:

```
main
```

Isso indica ao Go:

"Este código vai gerar um programa que pode ser executado."

Além disso, precisa existir uma função:

```
func main()
```

Ela será o ponto inicial do programa.

3. Os imports

O programa importa:

```
import (  
    "encoding/json"  
    "fmt"  
    "log"  
    "net/http"  
    "strconv"  
)
```

Cada pacote oferece funcionalidades diferentes.

encoding/json

Serve para trabalhar com JSON.

Por exemplo, transformar:

```
Product{  
    ID: 1,  
    Nome: "Mouse",  
    Preço: 120,  
}
```

em:

```
{  
    "id": 1,  
    "nome": "Mouse",  
    "preco": 120  
}
```

E também faz o contrário.

Recebe JSON:

```
{  
    "nome": "Webcam",  
    "preco": 299.90  
}
```

e transforma em uma struct Go.

fmt

Utilizado para mostrar informações no terminal.

Exemplo:

```
fmt.Println("Servidor iniciado")
```

log

Também mostra mensagens, mas é mais adequado para registrar acontecimentos do servidor.

Por exemplo:

```
log.Printf("Método: %s", r.Method)
```

Pode produzir:

2026/08/12 09:30:20 Método: GET

O log é útil porque informa o que está acontecendo dentro da aplicação.

net/http

É provavelmente o pacote mais importante desse programa.

Ele permite criar:

- servidor HTTP
- rotas
- handlers
- requisições
- respostas
- códigos HTTP

Tudo sem instalar nenhum framework externo.

strconv

Serve para converter valores.

Neste programa fazemos:

```
id, err := strconv.Atoi(idTexto)
```

Atoi significa aproximadamente:

ASCII to Integer

Ou seja:

"10" → 10

Isso é necessário porque parâmetros recebidos pela URL chegam inicialmente como texto.

4. A struct Product

Temos:

```
type Product struct {  
    ID      int      `json:"id"`  
    Nome    string   `json:"nome"`  
    Preço  float64  `json:"preco"`  
}
```

Essa estrutura representa um produto.

Podemos pensar em uma struct como uma ficha.

Por exemplo:

Produto

ID: 1

Nome: Notebook
Preço: 3500

Em Go podemos criar:

```
produto := Product{  
    ID: 1,  
    Nome: "Notebook",  
    Preço: 3500,  
}
```

Os tipos são:

ID int

Número inteiro.

Nome string

Texto.

Preço float64

Número com casas decimais.

5. O que significa json: "nome"?

Observe:

```
Nome string `json:"nome"`
```

Essa parte:

```
`json:"nome"`
```

é chamada de **struct tag**.

Ela informa ao pacote JSON qual nome deverá aparecer no JSON.

Sem isso poderíamos receber:

```
{  
    "ID": 1,  
    "Nome": "Notebook",  
    "Preço": 3500  
}
```

Com as tags:

```
ID    int    `json:"id"`  
Nome  string `json:"nome"`  
Preço float64 `json:"preço"`
```

teremos:

```
{  
    "id": 1,  
    "nome": "Notebook",  
    "preço": 3500  
}
```

6. A lista de produtos

O programa possui:

```
var produtos = []Product{
    {ID: 1, Nome: "Notebook", Preço: 3500.00},
    {ID: 2, Nome: "Mouse", Preço: 120.00},
    {ID: 3, Nome: "Teclado", Preço: 250.00},
}
```

Aqui temos um **slice** de produtos.

Um slice pode ser entendido inicialmente como uma lista.

```
produtos
|
+-- Notebook
+-- Mouse
+-- Teclado
```

O tipo:

```
[]Product
```

significa:

"uma coleção de elementos do tipo Product."

Importante: isso **não é banco de dados**.

Os dados estão somente na memória RAM.

Se o programa for encerrado:

```
Servidor desligado
      ↓
dados cadastrados desaparecem
```

Os três produtos escritos diretamente no código aparecem novamente quando o servidor reinicia.

7. A função `main()`

Aqui começa efetivamente a execução do programa:

```
func main() {
```

O código cria primeiro:

```
produtosHandler := http.HandlerFunc(produtosRouter)
```

Isso transforma:

```
produtosRouter
```

em um objeto que o servidor HTTP consegue utilizar como `Handler`.

Depois:

```
http.Handle(
    "/produtos",
    logging(produtosHandler),
)
```

Estamos dizendo:

"Quando alguém acessar `/produtos`, execute esse Handler."

Mas existe um detalhe importante.

Não executamos diretamente:

```
produtosHandler
```

Executamos:

```
logging(produtosHandler)
```

Isso significa que a requisição passa primeiro pelo middleware.

O fluxo fica:

```
Cliente
  ↓
/produtos
  ↓
logging
  ↓
produtosRouter
  ↓
GET / POST / PATCH / DELETE
```

Essa configuração está no início do programa.

8. Iniciando o servidor

O programa executa:

```
err := http.ListenAndServe(":8080", nil)
```

Essa é a linha que realmente inicia o servidor.

A porta escolhida é:

```
8080
```

Portanto podemos acessar:

```
http://localhost:8080/produtos
```

O `nil` significa que estamos usando o roteador padrão do próprio `net/http`.

9. Por que existe `err`?

Temos:

```
err := http.ListenAndServe(":8080", nil)
```

O servidor pode falhar.

Por exemplo:

```
porta 8080 já está sendo utilizada
```

Então:

```
if err != nil {  
    log.Fatal(err)  
}
```

quer dizer:

"Se houver algum erro, mostre-o e encerre o programa."

Esse padrão aparece o tempo todo em Go:

```
resultado, err := algumaFuncao()  
  
if err != nil {  
    // tratar o erro  
}
```

10. O middleware de logging

Temos:

```
func logging(next http.Handler) http.Handler {
```

Um middleware fica no meio do caminho entre a requisição e a função final.

Imagine:

```
Cliente  
  ↓  
Segurança  
  ↓  
Recepcionista  
  ↓  
Funcionário responsável
```

O middleware é parecido com essa recepcionista.

Ele pode:

- registrar acessos
- verificar usuário
- verificar token
- modificar cabeçalhos
- impedir uma requisição
- medir tempo

O middleware logging recebe:

```
next http.Handler
```

next significa:

"qual é o próximo Handler que deverá ser executado?"

Dentro dele temos:

```
log.Printf(
```

```
"Método: %s | Rota: %s",  
r.Method,  
r.URL.Path,  
)
```

Se alguém fizer:

```
GET /produtos
```

poderemos ver no terminal algo como:

```
Método: GET | Rota: /produtos
```

Depois aparece:

```
next.ServeHTTP(w, r)
```

Essa linha é extremamente importante.

Significa:

```
"Meu middleware terminou. Agora pode continuar."
```

Todo o funcionamento desse middleware aparece nessa seção do arquivo.

11. **w e r**

Você verá em quase todas as funções:

```
w http.ResponseWriter,  
r *http.Request,
```

Esses dois parâmetros são fundamentais em servidores Go.

r

```
r *http.Request
```

Representa a **requisição que chegou**.

Com ele conseguimos descobrir:

```
r.Method
```

Método:

```
GET  
POST  
PATCH  
DELETE
```

Também:

```
r.URL
```

URL acessada.

E:

```
r.Body
```


corpo enviado pelo cliente.

Podemos pensar assim:

```
r = aquilo que o cliente mandou
```

w

```
w http.ResponseWriter
```

É utilizado para montar a resposta.

Podemos pensar:

```
w = aquilo que o servidor vai responder
```

Por exemplo:

```
w.WriteHeader(http.StatusOK)
```

ou:

```
json.NewEncoder(w).Encode(produto)
```

Portanto:

```
Cliente → r → servidor
```

```
Cliente ← w ← servidor
```

12. Middleware de autenticação

Existe outro middleware:

```
func autenticacao(next http.Handler) http.Handler
```

Ele protege determinadas operações usando **Basic Authentication**.

Temos:

```
usuario, senha, ok := r.BasicAuth()
```

Essa linha retorna três informações.

```
usuario  
senha  
ok
```

Por exemplo:

```
usuario = "admin"  
senha   = "1234"  
ok      = true
```

Depois:

```
if !ok ||  
    usuario != "admin" ||  
    senha != "1234" {
```

Estamos dizendo:

```
SE
```

```
não existe autenticação
OU
usuário é diferente de admin
OU
senha é diferente de 1234
ENTÃO
bloqueeie.
```

13. O operador !

Observe:

```
!ok
```

O ! significa **negação**.

Se:

```
ok == true
```

então:

```
!ok == false
```

Se:

```
ok == false
```

então:

```
!ok == true
```

Por isso:

```
if !ok
```

significa:

```
"Se a autenticação não tiver sido enviada."
```

14. O operador ||

Temos:

```
if !ok ||
    usuario != "admin" ||
    senha != "1234"
```

|| significa:

OU

Basta uma condição ser verdadeira para entrar no `if`.

15. HTTP 401

Se a autenticação estiver errada:

```
http.Error(  
    w,  
    "Acesso não autorizado",  
    http.StatusUnauthorized,  
)
```

`http.StatusUnauthorized` corresponde a:

401 Unauthorized

Depois existe:

```
return
```

Muito importante.

Sem o `return`, o código poderia continuar executando mesmo depois da autenticação falhar.

16. O roteador

A função:

```
func produtosRouter(  
    w http.ResponseWriter,  
    r *http.Request,  
)
```

recebe todas as requisições feitas para:

`/produtos`

Ela então verifica qual método HTTP foi utilizado.

Isso é feito com:

```
switch r.Method
```

É parecido com vários `if`.

17. GET

Temos:

```
case http.MethodGet:  
    getProdutos(w, r)
```

Ou seja:

```
se método == GET  
    ↓  
executar getProdutos()
```

18. POST

Temos:

```
case http.MethodPost:

    autenticacao(
        http.HandlerFunc(postProduto),
    ).ServeHTTP(w, r)
```

Aqui existe um detalhe interessante.

A função:

`postProduto`

não é chamada diretamente.

Primeiro fazemos:

```
http.HandlerFunc(postProduto)
```

Depois:

```
autenticacao(...)
```

Então o fluxo fica:

```
POST /produtos
  ↓
autenticacao
  ↓
postProduto
```

19. PATCH

O mesmo princípio:

```
case http.MethodPatch:

    autenticacao(
        http.HandlerFunc(patchProduto),
    ).ServeHTTP(w, r)
```

Fluxo:

```
PATCH
  ↓
autenticação
  ↓
patchProduto
```

20. DELETE

Também:

```
case http.MethodDelete:

    autenticacao(
        http.HandlerFunc(deleteProduto),
```

```
).ServeHTTP(w, r)
```

Fluxo:

```
DELETE
↓
autenticação
↓
deleteProduto
```

21. Métodos não permitidos

Se alguém tentar:

```
PUT
HEAD
OPTIONS
```

cairá no:

```
default:
```

E será retornado:

```
http.StatusMethodNotAllowed
```

que representa:

```
405 Method Not Allowed
```

22. GET de todos os produtos

A função:

```
getProdutos()
```

permite duas operações.

Consultar tudo:

```
GET /produtos
```

ou consultar um produto:

```
GET /produtos?id=2
```

O programa começa fazendo:

```
idTexto := r.URL.Query().Get("id")
```

Por exemplo:

```
/produtos?id=2
```

teremos:

```
idTexto = "2"
```

Observe que ainda é uma string.

Essa lógica está implementada na função de consulta.

23. Quando não existe ID

Temos:

```
if idTexto == "" {
```

Isso significa:

```
"Se nenhum ID foi informado."
```

Então:

```
responderJSON(  
    w,  
    http.StatusOK,  
    produtos,  
)
```

retorna a lista inteira.

Exemplo:

```
[  
    {  
        "id": 1,  
        "nome": "Notebook",  
        "preco": 3500  
    },  
    {  
        "id": 2,  
        "nome": "Mouse",  
        "preco": 120  
    }  
]
```

24. Convertendo o ID

Se tivermos:

```
/produtos?id=2
```

o valor "2" precisa virar o número 2.

Usamos:

```
id, err := strconv.Atoi(idTexto)
```

Resultado:

```
id = 2  
err = nil
```

Mas:

```
/produtos?id=batata
```

produzirá erro.

Então:

```
if err != nil
```

e a API responde:

```
400 Bad Request
ID inválido
```

25. Procurando o produto

Temos:

```
for _, produto := range produtos {
```

Esse código percorre a lista.

Imagine:

```
Notebook
Mouse
Teclado
```

O for vai olhar:

```
Notebook
  ↓
Mouse
  ↓
Teclado
```

Para cada produto:

```
if produto.ID == id
```

Se encontrar:

```
responderJSON(
    w,
    http.StatusOK,
    produto,
)
```

26. O _ no for

Temos:

```
for _, produto := range produtos
```

O range normalmente pode devolver:

```
indice, valor
```

Por exemplo:

```
for i, produto := range produtos
```

Mas nesse caso não precisamos do índice.

Por isso usamos:

—

Em Go `_` significa:

"Estou recebendo este valor, mas não quero utilizá-lo."

27. Produto não encontrado

Se o `for` terminar sem encontrar nada:

```
http.Error(  
    w,  
    "Produto não encontrado",  
    http.StatusNotFound,  
)
```

Retorna:

404 Not Found

28. POST: criando um produto

A função:

```
postProduto()
```

espera receber algo como:

```
{  
    "nome": "Webcam",  
    "preco": 299.90  
}
```

Ela começa:

```
var novoProduto Product
```

Nesse momento temos aproximadamente:

```
novoProduto.ID      = 0  
novoProduto.Nome    = ""  
novoProduto.Preco   = 0
```

Depois o JSON recebido será colocado nessa variável.

29. `json.NewDecoder`

Temos:

```
err := json.NewDecoder(  
    r.Body,  
) .Decode(&novoProduto)
```

Podemos simplificar mentalmente para:

```
json.NewDecoder(r.Body).Decode(&novoProduto)
```


`r.Body` contém o JSON recebido.

Por exemplo:

```
{
  "nome": "Webcam",
  "preco": 299.90
}
```

`Decode()` transforma isso em:

```
novoProduto.Nome = "Webcam"
novoProduto.Preco = 299.90
```

30. Por que existe `&novoProduto`?

Essa é uma parte importante para quem está começando em Go.

Temos:

```
Decode(&novoProduto)
```

O `&` significa:

"endereço de memória de `novoProduto`."

Se passássemos apenas:

```
novoProduto
```

estariamos entregando o valor.

Quando passamos:

```
&novoProduto
```

permitimos que `Decode` altere a variável original.

Podemos imaginar:

```
novoProduto
  |
  | endereço
  ↓
0x1234
```

O `Decode` recebe esse endereço e escreve os dados naquele local.

31. Gerando o ID

Depois:

```
novoProduto.ID = len(produtos) + 1
```

Se temos 3 produtos:

```
len(produtos)
```

retorna:

3

Então:

$3 + 1 = 4$

O novo produto recebe:

`ID = 4`

Para um exercício didático funciona.

Mas existe uma pequena limitação: se produtos forem excluídos, `len(produtos)+1` pode eventualmente produzir IDs repetidos. Em uma aplicação real, normalmente deixaríamos o banco de dados cuidar da geração do identificador.

32. append

Para adicionar o produto:

```
produtos = append(
    produtos,
    novoProduto,
)
```

Podemos simplificar para:

```
produtos = append(produtos, novoProduto)
```

Antes:

```
Notebook
Mouse
Teclado
```

Depois:

```
Notebook
Mouse
Teclado
Webcam
```

33. HTTP 201

Depois o programa responde:

```
http.StatusCreated
```

que significa:

```
201 Created
```

É o código correto para informar:

"Um novo recurso foi criado."

34. PATCH: alteração parcial

O PATCH é uma das partes mais interessantes do programa.

Podemos mandar apenas:

```
{
    "preco": 150
}
```

Sem alterar o nome.

A função utiliza:

```
var dados struct {
    Nome  *string `json:"nome"`
    Preço *float64 `json:"preco"`
}
```

Observe os *.

35. Por que *string e *float64?

O problema seria diferenciar:

```
{}
```

de:

```
{
    "preco": 0
}
```

Se usássemos:

```
Preco float64
```

nos dois casos poderíamos terminar com:

```
0
```

Usando:

```
Preco *float64
```

podemos saber se o campo foi enviado.

Não enviado:

```
dados.Preco = nil
```

Enviado:

```
dados.Preco aponta para um float64
```

Por isso:

```
if dados.Preco != nil {
```

significa:

"Se o cliente realmente enviou preco."

36. O que significa `*dados.Preco`?

Temos:

```
produtos[i].Preco = *dados.Preco
```

Aqui o `*` tem outro uso.

Se:

```
dados.Preco
```

é um ponteiro, ele contém um endereço.

```
dados.Preco
  ↓
endereço
  ↓
150.00
```

Então:

```
*dados.Preco
```

significa:

"Quero o valor que está guardado naquele endereço."

Isso é chamado de **desreferenciar o ponteiro**.

37. Por que `for i := range produtos` no PATCH?

No GET tínhamos:

```
for _, produto := range produtos
```

Mas no PATCH temos:

```
for i := range produtos
```

Por quê?

Porque precisamos modificar o elemento original.

Podemos acessar:

```
produtos[i]
```

Por exemplo:

```
produtos[0] = Notebook
produtos[1] = Mouse
produtos[2] = Teclado
```

Então podemos fazer:

```
produtos[i].Preco = 150
```

e modificar o produto dentro do slice.

38. DELETE

A função:

```
deleteProduto()
```

procura primeiro o produto pelo ID.

Depois remove usando:

```
produtos = append(  
    produtos[:i],  
    produtos[i+1:]...,  
)
```

Essa sintaxe parece complicada inicialmente, mas a ideia é simples.

Suponha:

```
índice 0 = Notebook  
índice 1 = Mouse  
índice 2 = Teclado
```

Queremos remover:

```
índice 1
```

Pegamos o que vem antes:

```
produtos[:1]
```

Resultado:

```
Notebook
```

Depois o que vem após:

```
produtos[2:]
```

Resultado:

```
Teclado
```

Juntamos:

```
Notebook + Teclado
```

Resultado:

```
Notebook  
Teclado
```

39. O que significa . . . ?

Temos:

```
produtos[i+1:]...
```

O . . . diz ao append:

"Pegue individualmente todos os elementos desse slice e acrescente-os."

Sem ele, estaríamos tentando adicionar um slice inteiro como se fosse apenas um elemento.

40. HTTP 204

Depois do DELETE temos:

```
w.WriteHeader(  
    http.StatusNoContent,  
)
```

Isso representa:

```
204 No Content
```

Quer dizer:

"A operação foi executada corretamente, mas não há conteúdo para retornar."

É muito comum após um DELETE.

41. A função responderJSON

No final temos:

```
func responderJSON(  
    w http.ResponseWriter,  
    status int,  
    dados interface{},  
)
```

Essa é uma **função auxiliar**.

Sem ela precisaríamos repetir várias vezes:

```
w.Header().Set("Content-Type", "application/json")  
w.WriteHeader(http.StatusOK)  
json.NewEncoder(w).Encode(produto)
```

Então criamos:

```
responderJSON(...)
```

e reutilizamos.

Isso segue um princípio importante:

Evitar repetir código desnecessariamente.

42. interface{ }

O parâmetro:

```
dados interface{ }
```

significa aproximadamente:

"dados pode receber valores de diferentes tipos."

Por exemplo:

```
Product
```

ou:

```
[]Product
```

Por isso podemos usar a mesma função para:

```
responderJSON(w, http.StatusOK, produto)
```

e:

```
responderJSON(w, http.StatusOK, produtos)
```

Em versões modernas de Go também podemos escrever:

```
dados any
```

any é equivalente a:

```
interface{}
```

e costuma ser mais fácil de explicar para iniciantes.

43. Content-Type

A função executa:

```
w.Header().Set(
    "Content-Type",
    "application/json",
)
```

Isso informa ao cliente:

"O conteúdo que estou enviando é JSON."

Ou seja, a resposta HTTP terá algo semelhante a:

```
Content-Type: application/json
```

44. WriteHeader

Depois:

```
w.WriteHeader(status)
```

Define o código HTTP.

Por exemplo:

```
200 OK
201 Created
400 Bad Request
404 Not Found
```

45. Marshal e Encode

Finalmente:

```
json.NewEncoder(w).Encode(dados)
```

O Go pega:

```
Product{
    ID: 2,
    Nome: "Mouse",
    Preço: 120,
}
```

e envia:

```
{
  "id": 2,
  "nome": "Mouse",
  "preco": 120
}
```

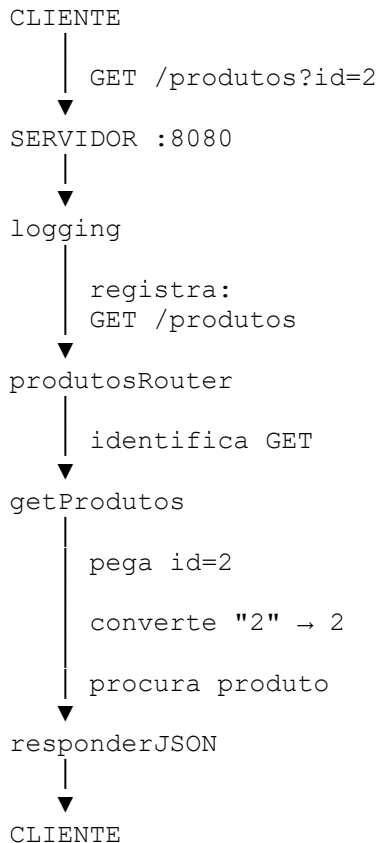
46. Visão completa do fluxo

A melhor maneira de um aluno iniciante entender esse programa é enxergar o caminho da requisição.

Quando fazemos:

```
GET http://localhost:8080/produtos?id=2
```

acontece:



Resposta:

```
{
  "id": 2,
  "nome": "Mouse",
  "preco": 120
}
```

Já um DELETE percorre:

```
CLIENTE
  |
  | DELETE /produtos?id=2
  ▼
logging
  ▼
produtosRouter
  ▼
autenticacao
  |
  | login errado → 401
  |
  | login correto
  |
  | ↓
  | deleteProduto
  | ↓
  | remove produto
  | ↓
  | 204 No Content
```

O ponto central para entendermos é que **cada requisição percorre uma cadeia de funções**. O `main()` monta essa cadeia, o `logging` observa a requisição, `produtosRouter` decide o que fazer e as funções `getProdutos`, `postProduto`, `patchProduto` e `deleteProduto` executam a operação propriamente dita.